

ПОЗИЦИЯ

3 ФЕВРАЛЯ 2026 ГОДА

Непрерывное обучение и эпоха постмолитного искусственного интеллекта

Стоимость непрерывного обучения возрастает по мере расширения возможностей.



Чарльз О'Нил (Базеттен)



Гарри Партридж (Бастетен)

Введение

Современные большие языковые модели страдают амнезией. Как только их контекстное окно заполняется, они начинают работу заново, ориентируясь только на краткое изложение. Это не ошибка, которую можно исправить, а скорее следствие того, как устроена память в трансформерах. По мере увеличения контекста кэш KV растет линейно, что приводит к быстрому потреблению большего объема памяти, чем все веса модели. Чтобы создавать системы, способные к непрерывному обучению, нам нужно перенести информацию с уровня токенов на уровень весов: из явного хранилища в суперпозицию.

Это подводит нас к выводу, который определяет дальнейшее развитие этой статьи: **непрерывное обучение неотделимо от специализации**. Затраты на запоминание зависят от объема информации, которую вы пытаетесь запомнить. Сузьте объем, и задача станет выполнимой. Если же информация будет слишком общей, модель начнет бороться с законами термодинамики.

Два типа специализации

Специализация может принимать две формы: специализация на задачах и специализация на предметной области. Большинство корпоративных сценариев использования ИИ сегодня сосредоточены на первом варианте — обобщении

документов, извлечении данных, классификации. Любая задача с фиксированным шаблоном запроса и четко определенным пространством вывода.

Мы считаем, что проблема непрерывного обучения для специализации задач практически решена.

Компании уже давно ведут непрерывный сбор данных. Современные стеки тонкой настройки позволяют легко настроить конвейер непрерывной тонкой настройки, независимо от того, используете ли вы обучение с подкреплением или самообучение с учителем. Кроме того, на практике нет большой разницы между десятью этапами тонкой настройки с 100 000 примеров на каждом или одним этапом с миллионом примеров. Если у вас достаточно данных, часто бывает достаточно одного обучающего цикла в начале. Если распределение данных меняется, вы переобучаете модель. Это инженерное дело, а не исследовательская работа.

Возникает интересный вопрос: что дает непрерывное обучение, помимо постепенного улучшения моделей для решения задач, с которыми они уже справляются? Какие новые сценарии использования становятся возможными?

Рассмотрим разницу между моделью, которая обобщает информацию из документов, и моделью, которая выступает в роли сотрудника. Почему мы не можем просто собрать записи разговоров между сотрудником и руководителем и обучить на их основе модель?

Проблема с сотрудниками усугубляется двумя факторами:

1. **Охват.** Для обобщения документов можно заранее определить набор критериев, которые охватывают задачу. Область применения ограничена. От сотрудника может потребоваться выполнение гораздо более широкого спектра задач, которые невозможно перечислить заранее.
2. **Эффективность выборки.** Модель для обобщения информации просматривает тысячи документов. У нее есть множество возможностей учиться на основе обратной связи. Сотрудник может получить инструкцию один раз и должен следовать ей всегда. У него нет второго шанса извлечь урок из повторяющегося примера.

Именно этот второй тип проблем — широкий охват, скудная обратная связь, кратковременная память — должен решаться с помощью непрерывного обучения. Именно на этом мы и сосредоточились в этой статье.

Фундаментальное противоречие

Лучше всего суть основной проблемы непрерывного обучения сформулировал Берен Миллидж.

Берен утверждает, что непрерывное обучение вступает в противоречие с долговременной памятью. Память не хранит информацию напрямую, а скорее содержит указатели на распределенные нейронные репрезентации. Когда вы что-то вспоминаете, вы активируете паттерн во множестве нейронов. Но непрерывное обучение постоянно обновляет эти самые репрезентации. Пути, на которые указывают ваши воспоминания, расходятся с теми, по которым они должны вести.

Это применимо к любой модульной системе. Если две подсистемы взаимодействуют посредством представлений, которые развиваются независимо друг от друга, их общий язык в конечном итоге перестает работать. Модуль А учится посылать сигнал X, означающий «опасность». Модуль В узнает, что X означает «опасность». Затем модуль А обновляется и начинает посылать сигнал Y. Но модуль В по-прежнему ожидает сигнал X, и интерфейс оказывается поврежден.

Мозг решает эту проблему с помощью заморозки интерфейса. После определенного критического периода (периода высокой нейропластичности в детстве) канал связи между системами становится фиксированным. Каждая подсистема по-прежнему может обучаться самостоятельно — она создает кодировщики и декодировщики, которые сопоставляют ее развивающиеся представления с замороженным каналом. Вот почему существуют критические периоды: они формируют стабильные интерфейсы, благодаря которым остальная система не дает сбоев каждый раз, когда вы изучаете что-то новое.

Но обратите внимание на то, что из этого следует. Если интерфейс заморожен, то все последующее обучение происходит *относительно этой замороженной базы*. База ограничивает то, что вы можете усвоить, и то, насколько эффективно вы можете это усваивать. Чем больше вы уже привязаны к базе, тем сложнее вам будет добавить что-то новое. Вот почему взрослым так сложно дается изучение языков, которые дети осваивают без труда: фонологический интерфейс застыл несколько десятилетий назад.

Вы можете возразить: взрослые учатся десятилетиями, ведь мы в целом остаемся умными людьми, и непрерывное обучение не приводит к катастрофическим последствиям.

Но люди гораздо более узкоспециализированы, чем большие языковые модели. Наши знания фрагментарны и разрозненны. Мы выбираем одну узкую область и углубленно изучаем ее. Хирург не обязан знать налоговое право, гидродинамику и китайский язык. Мозг решает проблему непрерывного обучения, изначально не стремясь к обобщениям.

Перед крупными лабораториями стоит более сложная задача: взять модель, которая уже все знает, и заставить ее учиться дальше, не забывая при этом то, что она уже усвоила.

Стоимость непрерывного обучения возрастает с увеличением охвата.

Модель, которая знает все, должна сохранять согласованность между всеми своими представлениями всякий раз, когда она узнает что-то новое. Каждое обновление может привести к сбоям в работе чего-то, не связанного с исходными данными. Можно разработать систему синхронизации, которая будет приводить сохраненные знания в соответствие с меняющимися представлениями, но это лишает долговременную память вычислительного преимущества, которое заключается в том, что ее можно не трогать, пока она не понадобится. Чем больше данных вы храните, тем сложнее их обслуживать.

У модели, которая знает только о страховых выплатах, такой проблемы нет, поскольку она может свободно обновляться, и ей почти не с чем взаимодействовать, почти не с чем синхронизироваться. После того как вы сформируете базовые представления, все новое обучение будет происходить относительно них. Чем больше вы вложили в модель, тем сложнее добавлять в нее что-то без ущерба для качества и тем дороже становится ее обслуживание.

Это принцип разделения труда, предложенный Адамом Смитом, применительно к нейронным сетям. У универсальности есть издержки, и они растут быстрее, чем линейно, по мере расширения охвата. Специализация полностью решает эту проблему. Модель медицинского кодирования создает интерфейсы, оптимизированные для медицинских концепций, и совершенствуется в рамках этого ограниченного пространства, где катастрофическое забывание становится локальной, а не глобальной проблемой, а затраты на синхронизацию остаются приемлемыми.

Невозможность непрерывного обучения универсальных моделей — это та самая невидимая рука, которая подталкивает к использованию узкоспециализированных моделей.

Некоторые очевидные и не столь очевидные идеи из научной литературы

Существует множество научных работ, посвященных непрерывному обучению. Большинство из них направлены на то, чтобы избежать описанного выше противоречия — найти какой-то способ обновлять представления, не нарушая работу интерфейса. Некоторые подходы весьма изобретательны. Некоторые могут

сработать в отдельных случаях. Но мы считаем, что ни один из них не позволяет избежать фундаментального компромисса.

Вот обзор идей, которые кажутся нам наиболее интересными, и объяснение, почему мы все еще настроены скептически.

Картриджи

На данный момент самый современный метод работы с конечными контекстными окнами заключается в том, чтобы дать модели команду «обобщить этот диалог». Все знают, что это не самый оптимальный подход, но найти решение получше на удивление сложно. Однако одно интересное наблюдение последних лет заключается в том, что многие инновации, появившиеся после обучения моделей, были основаны на возможностях, которые изначально были реализованы с помощью подсказок. Самый яркий пример — модели логического вывода: до появления o1 у нас была команда «Думай шаг за шагом».

What then is the equivalent generalization of ‘summarize this conversation’? One approach from last year is the idea of constructing ‘Cartridges’. The idea here is to construct a KV cache that efficiently compresses prior knowledge into a dense learned representation. This is done by prompting the model to ‘self-study’ the prior knowledge, and using back propagation to update the KV cache instead of the model weights.

In fact, this idea is quite old. In 2019, FAIR published Augmenting Self-attention with Persistent Memory. This paper highlights the following point: a trained KV cache is actually very mathematically similar to a multi-layer perceptron (MLP) layer. Indeed, taking a dot product with a set of keys is equivalent to multiplying with an up-projection matrix. The softmax is then a suitable activation function, and a linear combination of value vectors is mathematically equivalent to multiplying with a down projection matrix. For this reason, the authors of this paper propose doing away with the MLP entirely: we simply make use of two types of KV vectors – static and dynamic. This makes the problem of continual learning even more tantalizing: it makes you wonder how we might move dynamic KV vectors into the static KV cache.

State Space Models

Of course, it would be remiss of us not to mention state space models in this context. In 2025, models like Qwen-3-Next, Kimi-Linear, Minimax-M1 and Nemotron 3 Nano demonstrated that hybridizing state-space models with interleaved full attention layers could lead to competitive long context performance. However, these full attention layers are still critical, so the context window is still growing indefinitely.

Without a non-linearity between the keys and the values, we are skeptical that pure state space models like delta-net and mamba-2 will have the expressivity needed to compete

with full attention. In a KV cache, the exponential decay of softmax allows you to distinguish between adjacent keys with high fidelity; without it, interference is inevitable. Indeed, the 128 by 128 state matrices which dominate the state-space status quo can only accommodate 128 rank 1 updates before perfect recall becomes impossible. As such, we see such state space models as essentially a smoothed version of sliding window attention. Certainly, this is an interesting innovation, but not a silver bullet for infinite context.

Sparse Memory Fine Tuning

Another idea is from a relatively old [paper](#), Mixture of a Million Experts, which pushes MoE to its logical extreme: a million experts, each a single neuron.¹ When we read this paper, our first thought was “what if you had a mixture of a *trillion* experts, and continual learning was just updating the router for new tasks?” This is because, with a trillion experts, you basically have the primitives for all function composition, and your router can just expressively map to these. But this also felt a bit overkill and moving away from the representation-based world of Beren. It felt like we should still be routing to and updating meaningful latents, in some form, rather than very atomic functional primitives.

[Sparse memory fine tuning](#) makes this idea more explicit in an update rule. Given a memory layer (architecturally similar to mixture of a million experts), they ask: which slots should you actually update when learning something new? Their answer is TF-IDF scoring (*a simple term-importance heuristic*): rank memory indices by how specific they are to the new input versus a background corpus. This identifies indices that are activated by *this* knowledge but not by everything else, avoiding the general-purpose slots that would cause interference. This is Beren's framework quite explicitly: a frozen addressing mechanism plus a simple heuristic (update what's specific, leave what's general) that sidesteps the need to learn how to learn.

Nested Learning

There was a lot of hype around the (very lengthy) paper from [DeepMind on nested learning](#), which starts with the observation that architecture and optimization aren't separate things. Rather, they're the same concept operating at different timescales. A transformer's attention mechanism is associative memory updating every forward pass. The feedforward layers are associative memory updating at training time. Backprop itself is associative memory, mapping inputs to their prediction errors. Once you see this, the whole model becomes a hierarchy of nested optimization problems, each with its own update frequency. This leads naturally to what they call “continuum memory systems”: instead of a binary split between short-term (attention) and long-term (weights), you get a spectrum of memory modules updating at different rates. Their proof of concept architecture, Hope, implements this as a self-modifying recurrent network that can optimize its own memory through self-reference. From the Beren framing, this is another way of establishing stable interfaces: if different components update at different frequencies, the slow-updating

ones become the frozen channel through which the fast-updating ones communicate. You get modularity through temporal separation of learning rates. (This builds on Titans; see below.)

We don't know enough about this to make an informed comment, but it feels a bit flawed. We think that it introduces way too many manual and qualitative design choices that are difficult to justify exactly why you chose them (number of time scales, what the frequency of those time scales should be, etc.). To us, it seems much cleaner to have the minimal set of discrete components of the system to emulate our meta-learning algorithm (a long-term memory like parameter weights, a short-term memory like the KV cache, and a way to inject the relevant bits of short-term memory into long-term memory when we need to). But we could be wrong. Maybe this is a solution.

Surprise Based Learning

Speaking of “when we need to”, a lot of people think surprise-based learning is the way to go.² One recent [paper](#) we read on this was entropy-adaptive fine-tuning (EAFT). EAFT identifies confident conflicts, which are tokens where the model has low entropy (strong prior) but low probability (the training label contradicts that prior). These generate massive gradients that overwrite existing representations, causing forgetting. Their fix is to weight the loss by normalised entropy, suppressing gradients when the model is confident. This dramatically reduces catastrophic forgetting.

But I'm skeptical. The whole point of learning is sometimes you *need* to override a confident prior. The president of the United States changed from Obama to Trump. A fact you were confident about turned out to be wrong. EAFT treats all confident conflicts as damage to be avoided, but some of them are exactly the updates you want. The real problem is that standard architectures don't have a clean separation between “what I know” and “how I reason”, so updating one fact corrupts everything else. Suppressing those updates just means you never learn the new facts.

Test time Training

And then there's test-time training. One interesting paper we saw on this was [Titans](#). This takes a memory-centric view of sequence modeling. They say attention (due to its limited context but accurate dependency modeling) acts as short-term memory, while a neural network that learns to compress information into its weights can act as long-term memory. Their neural memory module is essentially a meta-model that learns how to memorize at test time, using gradient descent on an associative memory loss. The clever bit is their surprise metric: an event that violates expectations (high gradient) is more memorable, but they decompose this into past surprise (momentum) and momentary surprise (current gradient), which prevents the model from missing important information after a big surprising moment. They also add a forgetting mechanism via weight decay, which they show is actually a generalisation of the gating in Mamba and friends. This feels like (1) a

more principled surprise-based approach than EAFT, but simultaneously (2) re-deriving RNNs/LSTMs and the like, so we're not sure how we feel about it.

A related and recent [paper](#) takes a simpler approach: just keep training a standard transformer with sliding-window attention at test-time via next-token prediction on the context it's reading. The model compresses context into its weights rather than storing every key-value pair explicitly. To make this work well, they use meta-learning at training time to prepare the model's initialization for test-time training, so the outer loop optimizes for "how good will this model be after it's been updated on the test context". The tradeoff is worse needle in a haystack retrieval, which makes sense, as the whole point is lossy compression, not lossless recall. I actually don't mind this, as it feels a lot closer to what humans are doing. At the same time, I think we still need a lossless form of short-term memory that we can more efficiently swap to, without defaulting to test-time training.

There's also a bunch of other interesting related papers, whether it's about extending the effective context window (e.g. lightning attention or landmark tokens)³, or other architecture changes like Memformer, which are worth reading.

The final one goes back to CARTRIDGES, which feels very spiritually close to sparse memory fine tuning. Again, if you treat your bank of CARTRIDGES as an associative memory, and the pre-trained transformer backbone establishes the interface, and you figure out how to retrieve and update relevant CARTRIDGES continuously, then you've got Beren's system. But maybe you want to be able to slowly add CARTRIDGES back into the weights, which is a whole other question entirely.

The common thread

Every approach here is trying to answer the same question: how do you update a neural network without breaking what it already knows?

The answers cluster into two families. Either you sparsify the update (million experts, TF-IDF routing, low-rank adapters), or you separate timescales (nested learning, test-time training, CARTRIDGES as a separate memory system). Both are ways of limiting interference. Neither escapes the fundamental tension.

Sparsification works when knowledge is separable. But the whole power of distributed representations is that they're *not* separable: features are reused across contexts, and that reuse is what gives you generalization. The more you sparsify, the more you're fighting the architecture.

Temporal separation works when you can afford to freeze something. But what you freeze becomes load-bearing. If you freeze too early, you're stuck with bad representations. If you freeze too late, you've already caused interference. And the thing you froze can never improve.

None of this means these techniques are useless. For narrow domains with clean separation, sparse updates might be enough. For applications where you can tolerate lossy compression, test-time training might work. But for the Big Lab dream, a general model that learns everything, forever, without forgetting anything, we don't see an escape hatch here.

Why this is probably the wrong framing

Most of the ideas in the previous section share an assumption: that the right way to solve continual learning is to learn how to learn. To us, it is fairly clear that humans don't *learn* their meta-learning algorithm. Instead, nature endows us with a relatively fixed, heuristic-based strategy for absorbing information: we learn our encoding during childhood, then use that mostly-frozen encoding to update our associative memory for the rest of our lives. If we also had to learn *how* to learn, we'd be dead long before we learned anything useful. (There's also evidence for this in animals. Zebras walk within minutes of birth. They're not figuring out locomotion from first principles.)

As Rich Sutton points out, evolution solved the meta-learning problem over tens of thousands of years. The important takeaway is that we don't have to. If we know what the meta-learning strategy is supposed to do, we don't need gradients on gradients. We don't have to bitter-lesson our way to the right algorithm. If we can design a system that (1) establishes an interface of representations plus encoders and decoders that read from and write to memory, and (2) slowly adds to that memory without disrupting the frozen interface, then we can just throw backprop at the subproblems. Learning representations and learning encoders/decoders are things we already know how to do. Retrieval might be as simple as vector search. Updates might be as simple as TF-IDFing which slots to touch, just like sparse memory finetuning.

And whilst we don't know enough about the human brain to be able to emulate *exactly* what this meta-learning strategy might be, I imagine it could be akin to the airplane emulating the flight of birds. We know what we need to do and what the end goal is (keeping a body in the air for an extended period of time), so we'll probably end up just figuring out what the end mechanism we need for this system, and writing some code/designing an architecture+algorithm to do it.

The Cambrian Zoo

So what does the future actually look like?

We think you'll see a proliferation of specialized models (thousands, eventually millions) each optimised for a narrow domain and continuously improving within it. Medical models that know medicine. Legal models that know law. Models fine-tuned on individual users' preferences, updating constantly.

These models won't share weights. They'll share APIs. The "general intelligence" emerges from composition rather than from cramming everything into one network. A routing layer (itself probably a model) will decide which specialist to invoke. The specialists can be updated, swapped out, improved independently. No global synchronization is required.

This is the Cambrian explosion that followed the Ediacaran period, a riot of specialized forms filling every niche. The foundation model era was Ediacara: a few general-purpose architectures dominating because nothing else had evolved yet. What comes next is adaptive radiation.

We don't think the Big Labs will stop trying to build god-models. There's too much momentum, too much narrative investment in AGI as a single artifact. But we think the actual deployed systems that matter, i.e. the ones doing useful work, making money, and improving over time, will be specialists. The monoliths will be impressive demos. The zoo will be the product.

The infrastructure for a world of specialized models looks different from the infrastructure for a world of monoliths. You need to serve thousands of models efficiently, not one model at massive scale. You need to route between them intelligently. You need to update them continuously without downtime. You need to version them, A/B test them, roll them back when something breaks.

This is what we're building toward at Baseten.

Appendix: RL vs SFT

No continual learning discussion would be complete without a discussion of RL vs SFT. The catastrophic forgetting literature has long treated forgetting as an architectural problem i.e. something to be solved with replay buffers, elastic weight consolidation, or careful regularisation. But recent work suggests the training objective itself might be the culprit. Specifically: SFT and RL have fundamentally different relationships to the model's existing knowledge, and this difference is best understood through the lens of KL divergence.

SFT minimizes negative log-likelihood over a dataset, which is equivalent (up to a constant) to minimizing the forward KL divergence between the data distribution and the model. This is a mode-covering objective. The model is heavily penalized for assigning low probability to any completion found in the training data; the loss increases exponentially as probability approaches zero. To avoid this penalty, the model must spread its probability mass across all modes in the dataset. The practical consequence is that when you fine-tune on new data, the model aggressively shifts probability toward that data, often at the expense of previously learned modes that aren't represented in the current batch.

RL, by contrast, maximizes rewards on completions sampled from the model's own policy (with KL regularization to the reference model). This is equivalent to minimizing the

reverse KL divergence (a mode-seeking objective). The model emphasizes high-reward outputs even at the cost of ignoring some output modes entirely. Importantly, assigning near-zero probability to some completion simply prevents it from being sampled; there's no exponential penalty forcing the model to cover that mode. The model can sharpen around what works without being dragged toward what the data says it should be doing.

This distinction turns out to be predictive of forgetting behavior. A recent group studying multimodal continual post-training, found that sequential SFT on seven tasks produced significant forgetting while reinforcement fine-tuning on the same sequence preserved prior-task performance almost entirely, approaching the upper bound of multi-task training without replay. Other papers report that RL achieves comparable target-task gains with substantially less degradation on non-target tasks. Another paper proposes what they call "RL's Razor": among all parameter configurations that solve a new task, online RL tends to converge to the one closest (in KL) to the original model. SFT, by contrast, can converge to solutions arbitrarily far from the base model depending on the training labels.

The common thread is on-policy learning. Because RL samples from the model's current policy, it trains on completions the model already assigns reasonable probability to. This implicitly preserves prior modes as you're reinforcing behaviors the model can already produce, not overwriting them with behaviors from an external distribution. The mode-seeking property of reverse KL means the model doesn't need to spread probability mass to match some target distribution; it can simply sharpen around what's working while leaving the rest of its knowledge largely untouched.

This suggests an important reframe: the problem with SFT isn't just that it's off-policy in the RL sense, but that it's mode-covering in a way that actively redistributes probability mass away from prior knowledge. Every batch of new data pulls the model toward full coverage of that batch's modes, creating interference with everything else. There's also some stuff which shows low-rank updates are important for less forgetting, with everything from LoRAs to RL.

The on-policy distillation work from Thinking Machines makes this concrete. They show that even training on a model's own samples via SFT degrades performance. Any finite batch exhibits distributional drift from the true policy, and the mode-covering objective amplifies this into progressive forgetting. On-policy distillation sidesteps this because the objective (reverse KL to a fixed teacher) is mode-seeking: the student converges on the teacher's behavior without the self-reinforcing drift of off-policy training. This is why they can recover instruction-following capability after mid-training on domain data, and why distillation is emerging as a tool for continual learning more broadly.

¹ Product key retrieval makes routing tractable; decomposing keys into a Cartesian product reduces complexity from $O(N)$ to $O(\sqrt{N})$.

² See everything from Friston’s free energy principle/predictive coding to Bayesian surprise to prioritised experience replay to Titans.

³ What if we could have some RL to earmark or tag bits we want to remember and replay, and incorporate into our long-term knowledge?

Other research



POSITION

The Michael Scott Paper Company of AI

Open-source doesn't need to wait out the frontier labs. It just has to wait out the gap.

 CHARLES O'NEILL

POSITION

If we can't design neat latent structures, then maybe we can Bitter Lesson it through self-study

They very existence of Claude's Constitution shows that self-study, to a certain extent, just works.

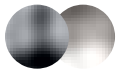


CHARLES O'NEILL

POSITION

A letter to the C-suite: the shifting role of MLEs

Your MLEs are brilliant, but you're giving them the wrong job.



CHARLES O'NEILL • 1 other

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Продукт

Специализированное логическое устройство

API-интерфейсы моделей

Обучение

Frontier Gateway

Платформа логического вывода Baseten

Исполняемые среды моделей

Инфраструктура

Мультиоблако

Варианты развертывания



Разработчик

Библиотека моделей

Документация

Список изменений

Ресурсы

Исследования

Клиенты

Блог

Руководства

Мероприятия

Партнеры

Калькулятор сбережений

Гибридное

Встраиваемая инженерия

Инженеры , развернутые на переднем крае

Модальности

Транскрипция

Генерация изображений

Преобразование текста в речь

Большие языковые модели

Комплексный ИИ

Эмbedding

ОТРАСЛИ

Предприятие

Здравоохранение

Стартапы

Вакансии

Контакты

Популярные модели

GLM 5.2

Kimi K2.7 Code

DeepSeek V4

GPT OSS 120B

Whisper Large V3

NVIDIA Nemotron 3 Ultra

Посмотреть все

Юридическая информация

Правила и условия

Политика конфиденциальности

Соглашение об уровне обслуживания

